TRUSTWORTHY AI AND DATA GOVERNANCE HACKATHON

Agentic Assurance

The Quest for Proof

DEVELOPER MASTER CLASS

accenture

MASTER CLASS

Agenda

Four sections. By the end you will know the concepts, exactly what to build, and exactly how you will be judged.

A	**What is Governance, Risk and Compliance**	The concepts: instruments, controls, claims, tests and evidence	**Accenture**
B	**What are the architecture patterns and best practices**	Where controls act, how they run, how evidence binds	**Accenture**
C	**What do you need to do in this Hackathon**	Your claim, your three objectives, and a worked example	**Accenture**
D	**What does it take to win**	The judging criteria, in full, published on day one	**Accenture**

TRUSTWORTHY AI AND DATA GOVERNANCE HACKATHON

A. What is Governance, Risk and Compliance

Dr. Emmanuel Munguia Tapia
Managing Director, AI · TAID Governance Workstream, Chair

accenture

WHY GOVERNANCE, RISK AND COMPLIANCE

AI has started doing things, not just saying things

When software does something, somebody eventually asks who allowed it. For almost every AI system running today, nobody can say for sure.

It answers a question

A model gives a wrong answer. Somebody reads it, shrugs, and asks again. The damage stops there.

It takes an action

An agent opens a ticket, changes a configuration and spends money on your behalf. The damage is already done.

THE AUDIT QUESTION

“Something went through our AI system on Tuesday morning. Can you show me what it did, which controls applied, and who was told when something went wrong?”

WHAT GOVERNANCE ACTUALLY IS

Four questions, and somebody has to answer all four

Governance, Risk and Compliance is not another system to install. It is the work of answering these questions and being able to show your answers.

01 POLICY	02 PROCESS	03 PLATFORM	04 PEOPLE
What must be true?	**How is it applied?**	**How is it proven?**	**Who is accountable?**
The claim you are making, its scope, and the numbers that define acceptable.	The objectives, the risks, the controls that hold them down, and the tests that check.	The records: events, measures, usage and decisions, joined so they can be queried.	A named owner for the claim, a team that builds it, and a reviewer who is not either.

Decide what good looks like › Build the trail that shows whether it happened › Demonstrate the result

Management owns the claim
not the team that builds it, and not the tool

Developers make it measurable
they choose the numbers and write them down first

Assurance checks the evidence
and the person who ran the test is not the person judged by it

AI INSTRUMENTS AND CONTROLS

Law says what. A control says how.

An AI instrument is any published document that sets expectations for how AI is built and used. The more binding it is, the less it tells you to do.

MOST BINDING AT THE TOP · MOST USEFUL AT THE BOTTOM

Law
EU AI Act · GDPR · national statutes
Binding. It carries penalties.

Official guidance
Commission guidelines · opinions
Not law. Follow it, or record why not.

Standards
CEN-CENELEC JTC 21 · ISO/IEC 42001
Voluntary, but they become the accepted way.

Frameworks
NIST AI RMF · CSA AICM · OECD
Nobody enforces them. Your best drafting source.

SO SOMEBODY HAS TO WRITE THE 'HOW'

A control is something you do, that somebody else can check

- **Documented**, in writing, independent of whoever does it today
- **Owned**, by a named person. Not a team, not a committee
- **Specific**, a stranger could tell whether it held last week

✗ **"Our AI behaves responsibly"**
Nothing to measure. Nobody owns it. No record could settle it.

✓ **"Any change beyond a routine one is approved by a named person, and the work stops if it is not"**
Either the approval is on record, or it is not.

HOW CONTROLS RUN

Five ways a control gets checked

What makes the check run, and whether it can stop anything. Real controls from the EU AI Act library, including the three you will build.

Static	Scheduled	Event driven	In transaction	Continuous
Somebody reads a document, occasionally	A job runs on a fixed cadence	Something happens and the check fires	Checked mid-action, before it completes	A number watched against a limit, always
AIA-GOV-001	AIA-INV-001	7 · AIA-LOG-001	16 · ACN-COST-001	9 · AIA-ARC-006
AI policy approved by top management	**Complete AI system inventory maintained**	**Automatic recording of events**	**Per-run spend cap**	**Production performance & drift**
Reviewed annually and on material change.	Quarterly reconciliation against what is actually running.	Every meaningful action emits a record, joined on one run id.	The gateway asks before the call, and can halt the run.	Live measure against a frozen baseline, alert to a named owner.

AND WHERE THE THRESHOLDS LIVES · CONTROL 9 AS THE EXAMPLE

AIA-ARC-006 *the parent, from the library*

"Monitored in production against defined thresholds, with alerts routed to a named owner." No metric. No number.

›

AIA-ARC-006.01 *the child, authored by you*

Diagnostic response score, baseline 0.900 frozen, relative drop ≤ 10% per window, 0 violating windows. Owner named. Version G-FM-v1, dated.

THE GRC CHAIN

From a claim to evidence somebody can check

Every word on this slide is vocabulary an auditor will use, and the spine that everything else hangs from.

CLAIM
what management asserts is true

›

OBJECTIVE
the measurable commitment that supports it

›

RISK
what goes wrong if it is not met

›

CONTROL
the safeguard you put in place

›

CONTROL TEST
the check that the safeguard worked

›

EVIDENCE
the records that make the test reproducible

WHEN A TEST FAILS, THE CHAIN LOOPS BACK ON ITSELF

Finding
raised, and it names whether the deficiency is in design, in operation, or both

›

Risk exposed
the residual rating goes back up to the inherent rating

›

Remediation
fix it, compensate for it, or accept it with a named approver and a hard expiry

›

Re-test
the loop closes here and nowhere else

AND FOUR ROLES, BECAUSE NOBODY MAY CHECK THEIR OWN WORK

Accountable management
owns the claim, and owns the response when it fails

GRC
defines what is required, what could go wrong, and which commitments apply

Operations
builds and runs the system and its controls, and produces the records

Assurance
applies the tests, and explains the results and the gaps

EVIDENCE

A log and an evidence trail are not the same document

An evidence trail is a set of records kept so that somebody who does not trust you can still rely on them.

	A LOG [Records Activity]	EVIDENCE TRAIL [Records Decisions]
Written for	An engineer hunting a bug next week	A sceptical stranger, years later
Completeness	Partial is fine	Complete, for the actions that matter
Editing	Fine. Nobody minds.	Impossible to change quietly
Kept for	Weeks	Years, on a stated retention rule
Proves who acted	Not really	Yes, and they cannot deny it

WHAT TURNS A RECORD INTO EVIDENCE FOR A CONTROL

- **what happened**
- **which control**
- **which threshold version**
- **which run**
- **who acted**
- **a seal**

The test: hand a stranger one record and its seal, and nothing else. If they can check it without your help, you have evidence. If they need you in the room, you have a database you are asking people to trust.

THRESHOLDS

Control have no threshold range. You have to choose it.

The law states an outcome and a reusable control fixes the shape of the answer. Only your own specification holds a number — and its date is what makes it a test.

The law *the legislature*	“An appropriate level of accuracy, declared in the instructions for use.”	NO NUMBER
The library control *whoever writes the library*	A metric exists, is justified against the intended purpose, and the achieved level is declared.	STILL NO NUMBER
Your specification *you*	Groundedness on the fixed test set, between [0.85, 1], measured on data held out from the last four quarters.	YOUR NUMBER
The test record *the tester*	Measured 0.81 on 12 March. Fail. Finding raised, owner assigned, re-test scheduled.	THE ANSWER

FOUR QUESTIONS FOR EVERY THRESHOLD RANGE

1. **Why this metric?**
   Tie it to the purpose and to the harm you are trying to avoid.
2. **Why this value?**
   A human baseline, a prior system, an industry reference. “It is what we achieved” is not an answer.
3. **Measured on what?**
   Name the dataset, the period, and how it was held out.
4. **When does it stop holding?**
   Work out the boundary conditions now, or you cannot disclose them later.

A threshold range chosen after seeing the result is a description of what you got, not a test. The version date on your threshold document is how anybody tells the difference.

TRUSTWORTHY AI AND DATA GOVERNANCE HACKATHON

B. Architecture Patterns and Best Practices

Dr. Emmanuel Munguia Tapia
Managing Director, AI · TAID Governance Workstream, Chair

Producing evidence is not "call a logging API". Five problems decide whether you built an architecture or a demo.

accenture

ENFORCEMENT POINTS

A control acts only where something passes through

There are many places where decisions are taken and things can go wrong. It is about position, not purpose. A component is not an enforcement point, a passage is, and nobody has to have built it for that reason.

✕ **A dashboard**
Sees everything. Can refuse nothing.

✓ **A gateway every request must cross**
Can look, can record, or can say no.

YOU HAVE FAR MORE OF THEM THAN YOU THINK

- **Model gateway** — every model request
- **Zone / API gateway** — each boundary crossing
- **Guardrails** — prompt in, answer out
- **Tool call** — the moment it does something
- **Retrieval and data access** — what it is allowed to read
- **Sub-agent handoff** — work passed to another agent
- **Human approval queue** — where a person decides
- **Planner** — a whole plan, before it runs
- **Agent memory write** — what it keeps between sessions
- **Deferred action** — queued now, runs later
- **Platform automation** — flows and custom code
- **Kill switch** — everything, at once

EVERY ENFORCEMENT POINT DOES THE SAME FIVE THINGS

1 Ask if this is allowed · 2 Wait for the answer · 3 Act on it · 4 Record what happened · 5 Seal the record

COMPLIANCE TIMING

Before it runs, while it runs, after it ran

The same control can be checked at three moments. Each needs different machinery and none substitutes for another.

BEFORE IT RUNS *ex ante*	WHILE IT RUNS *in the path*	AFTER IT RAN *ex post*
Pre-deployment compliance *also called deployment-time governance, design-time assurance, or ex-ante conformity* **"May this go live at all?"** • Offline evaluation on a fixed, held-out dataset • Policy-as-code checks in the build pipeline • A release or admission gate that can refuse • Signed model, prompt and threshold bundle • Documented sign-off against the control set **Output: go or no-go, once.**	**Continuous compliance** *also called runtime governance, enforcement at the moment of action* **"Is this allowed, right now?"** • Enforcement point in the request path • Evaluator returning a verdict in milliseconds • Guardrails on the prompt in and answer out • Rolling metric window against a threshold • Alert routed to a named owner who responds **Output: allow, deny or alert, per action.**	**Post-run compliance** *Forensics, reconstructing what happened from the records you kept* **"Did the control hold between T1 and T2?"** • A query tool over the evidence ledger • Reconciliation: expected records against observed • Recomputing the metric for the closed window • Trace reconstruction for a single case • A finding, a remediation task, a passing retest **Output: a dated pass or fail on the control.**

THE TWO THINGS PEOPLE CONFUSE IN THE MIDDLE COLUMN

EVALUATOR *judges the subject*	TESTER / THE CONTROL TEST *judges the control*
Asks is this one action, or this one measurement, inside the rule?	**Asks** did the control hold for everything in the window?
Output a verdict the system acts on: allow, deny, alert	**Output** a dated pass or fail, with measured values and exceptions
Consumer the enforcement point, in milliseconds	**Consumer** the register, the auditor, the judge

WHAT MAKES THIS A HACKATHON

Five problems, and none of them is logging

Recording evidence is the easy part. These five decide whether you built an architecture or a demo.

1 Controls run in five modes

One definition and one evidence format, whether the check runs inline in 5 ms or overnight in a batch.

- **Static** — read occasionally
- **Scheduled** — a job on a clock
- **Event driven** — fires on a change
- **In transaction** — the only one that prevents
- **Continuous** — a rolling window

Everything else reports after the fact.

2 How does an enforcement point invoke you?

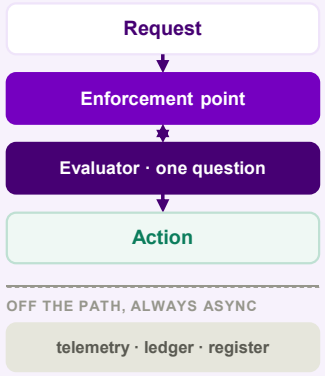
Define the interface once, so a new enforcement point can be added without a rewrite.

- **evaluate( )** — synchronous, in the path
- **emit( )** — event bus, asynchronous
- **callback( )** — webhook, verdict arrives later
- **pull( )** — on a schedule, off the path
- **sidecar / SDK** — interceptor in the process

If the second one forces a rewrite, you built an integration.

3 What is allowed in the path of execution, to avoid race conditions?

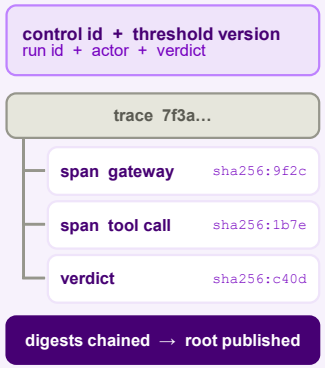
Only the enforcement point may make the work wait, and only for one question.



Race: two calls both read 9,800 under a 10,000 cap and both pass. Reserve, then commit.

4 How a record becomes evidence

A span is not evidence. It has to be bound to a control, a version and a run.



[OUT OF SCOPE]

Each producer signs its own statement. Change one byte and the chain breaks.

5 Does it cover every kind of enforcement point?

Some cannot be modified, some answer late, some have no request path at all.

- **Managed gateway** — no code of yours in it, read its log
- **Human approval queue** — the verdict arrives hours later
- **Scheduled batch job** — no request to sit inside
- **Third-party SaaS action** — no hook, poll and reconcile
- **Kill switch** — out of band, and last to survive

One design has to reach all of them.

A PATTERN TO START FROM

Separate the decision from the enforcement

A starting point, not an answer. Draw your own version on a whiteboard in the first hour and show it to a mentor.

THE LIVE PATH · ALREADY RUNNING BEFORE YOU ARRIVE

AI system
the agents at work

›

Enforcement points
can watch, or refuse

›

Telemetry
what the system emits

THE GOVERNANCE SIDE · WHAT TURNS SIGNALS INTO EVIDENCE TIED TO A CONTROL

Control register
controls, thresholds, owners, versions. Hands out the current version. Never in the path.

Evaluator
compares a value against its threshold range and returns a verdict at the decision point. Stateless.

Evidence ledger
binds a record to control, version and run, then seals and chains it. [Seal is out of scope]

Control test / query
answers “was control X satisfied between T1 and T2?” on demand.

ONE CONTRACT, FOUR TRANSPORTS

Every enforcement point reaches your architecture the same way. This is the part worth getting right.

evaluate( )
synchronous, in the path — for in-transaction controls that must be able to refuse

emit( )
asynchronous, on an event bus — for everything that only has to be recorded

callback( )
a webhook for a verdict that arrives later — deferred actions, human approval

pull( )
on a schedule — for scheduled and continuous controls, off the live path

FIVE HABITS THAT KEEP THE DESIGN HONEST

- **The telemetry store never judges**
- **The evaluator never remembers**
- **The control register is never in the path of execution**
- **The evidence ledger never interprets**
- **Records are sealed where they are made [out of scope]**

TRUSTWORTHY AI AND DATA GOVERNANCE HACKATHON

C. What do you need to do in this Hackathon

Accenture

One claim. Three objectives. Your numbers, your controls, your control tests.

accenture

THE LAB

One customer, one fault, three agents

A customer’s internet keeps dropping. Three agents collaborate across three zones. There is no direct customer-to-network route.

Customer zone

Chat agent
Trouble ticket

SERVICENOW

GATEWAY A

IT zone

Triage agent
Account records

AWS

GATEWAY B

Network zone

Investigation agent
Network inventory, digital twin

AWS · NVIDIA

SHARED MODEL GATEWAY — Every model call from every zone crosses it · token usage and budget decisions · where a run can be halted

THE LAB PROVIDES

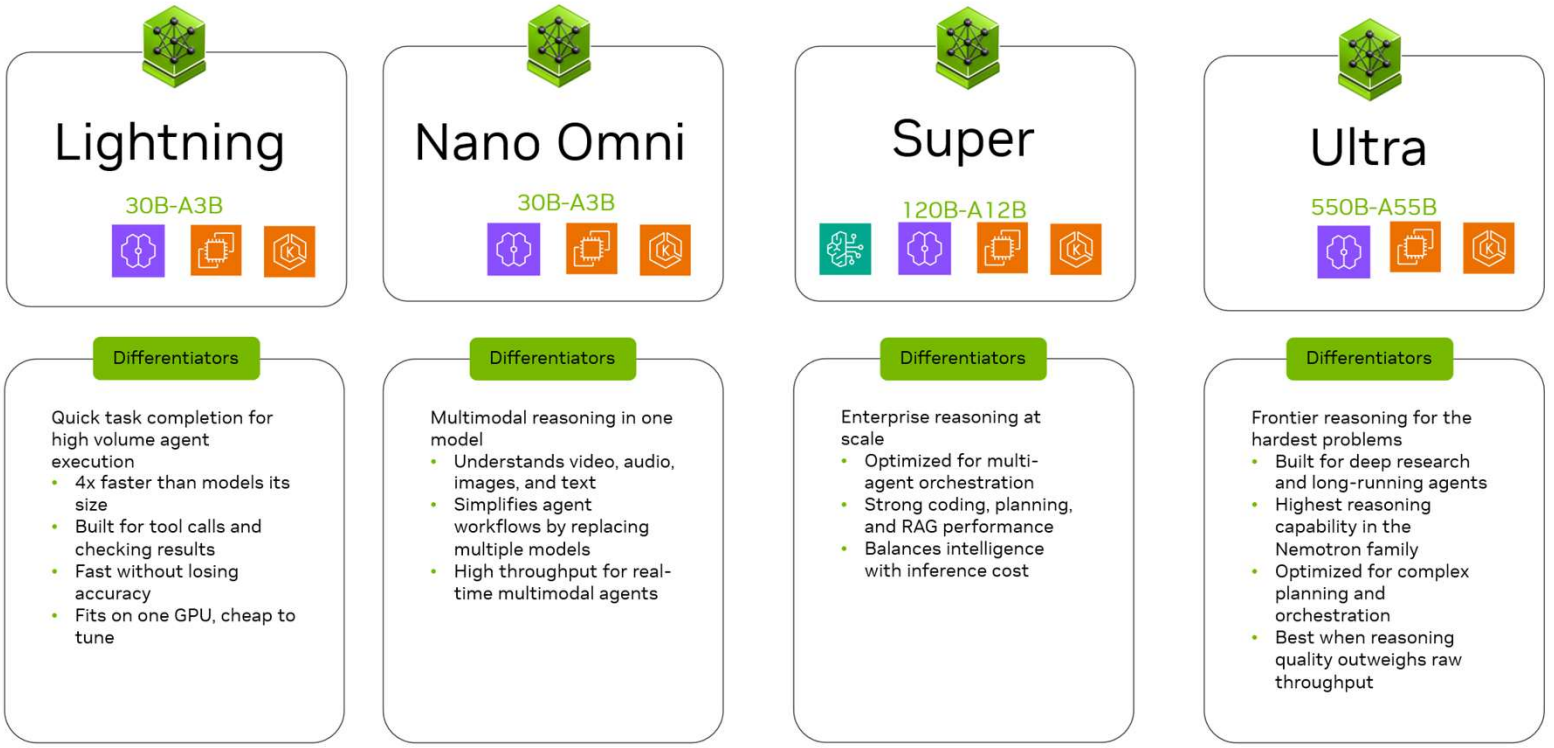
- Three agents, two zone gateways, one shared model gateway
- Guardrails on the prompt in and the answer out
- An OpenTelemetry collector and storage to keep what arrives
- A digital twin, a scenario switch, sandboxes and credentials

YOU BUILD

- Your thresholds, declared and dated before the assessed run
- The controls, at whichever enforcement points you choose
- The evidence trail that binds each record to a control
- The control tests, and whatever tool answers our questions

THE MODELS

NVIDIA Nemotron: A Family of 3 Open, Efficient and Intelligent Models



YOUR GOVERNANCE STRUCTURE

One claim. Three controls. Your threshold ranges.

Everything you build hangs off one management claim. Each control is a branch, and each branch you evidence is more of the claim answered.

CLAIM C-FM-01 — **“During fault resolution, the three collaborating agents meet our declared event-recording, drift and token-spend requirements.”**

7 AIA-LOG-001

Automatic event recording

Event driven

Every in-scope event recorded and joined to one run, across all three zones.

MEASURE: expected-event coverage, and the gap between consecutive events.

YOU SET: the coverage target, the gap limit, and the tolerated exception rate.

9 AIA-ARC-006

Drift and performance

Continuous

A live measure watched against a frozen baseline, with a named owner who responds.

MEASURE: relative drop from the baseline, per comparable window.

YOU SET: the metric, the baseline, the drift limit and the violating-window allowance.

16 ACN-COST-001

Per-run spend cap

In transaction

Token spend summed across all three agents and every retry, for the whole run.

MEASURE: input plus output tokens, reconciled per run.

YOU SET: the cap, and whether any over-cap run is permitted at all.

THE GOVERNANCE STRUCTURE

Turn the claim into a checkable tree

Claim → objective → risk → control test → evidence. One branch per objective, and your team supplies every number in square brackets.

CLAIM C-FM-01 — **“During fault resolution, the collaborating agents meet our declared event-recording, drift and token-spend requirements.”**

	7 Record the events AIA-LOG-001	9 Monitor drift AIA-ARC-006	16 Limit token spend ACN-COST-001
CONTROL OBJECTIVE	All events recorded: coverage [C%]. Gap < [G ms]; breaches ≤ [B7%].	Drift ≤ [D%] against the baseline. Violating windows ≤ [B9%].	All agents plus retries, per run: total spend ≤ [N tokens].
RISK	Missing events, broken run links, or excessive unexplained gaps.	Drift goes unnoticed, or the measurements themselves are missing.	A loop overspends, or calls are missed or charged to the wrong run.
CONTROL TESTS	Reconcile the expected records. Calculate gaps; count breaches.	Recalculate drift; count breaches. Check monitoring and alert response.	Reconcile usage; sum each run. Compare with cap; check stop records.
EVIDENCE	Expected IDs, events, timestamps and run links.	Baseline, live values, windows, Alerts, owner and the required response.	Call IDs, token counts, budget version and stop records.

A control is the safeguard. A control test checks it. Results flow back up to the claim, and a completed subset is partial coverage, not a pass.

WORKED EXAMPLE

Control 16, Per-run spend cap, at the shared model gateway

One enforcement point, one control, one threshold range, one question a judge can ask. Yours will differ, the shape should not.

1	DECLARE	**10,000 tokens per fault-resolution run**, all three agents and every retry included, zero over-cap runs permitted. Frozen as G-FM-v1 and dated before the first assessed run.
2	ENFORCE	**The shared model gateway is the enforcement point**: every model call from every zone crosses it. On each call it asks the evaluator "is run FM-A still under cap?" and acts on the verdict.
3	RECORD	**The gateway emits one record per call** to your ledger, asynchronously, off the request path, bound to the run, the control and the threshold version.
4	TEST	`controltest 16 --run FM-A` **reconciles every call for the run, sums input and output tokens, compares against the cap** named in G-FM-v1, and returns a verdict with the call identifiers behind the number.
5	SHOW	**FM-A 9,600 satisfied · FM-B 10,800 not satisfied**. One of two runs over cap, none permitted, so objective 16 is not satisfied — and the 800-token overshoot stays visible.

ONE EVIDENCE RECORD

run.id	**FM-A**
call.id	**c-0042**
control.id	**ACN-COST-001**
threshold.version	**G-FM-v1**
threshold.value	**10000**
measured.value	**9600**
verdict	**SATISFIED**
actor.id	**investigation-agent**
emitted.at	**2026-09-30T09:14:02Z**
seal	**sha256:9f2c…**

THE QUESTION A JUDGE WILL ASK

"Show me control 16 for run FM-B."

Two minutes, from your tool rather than from your memory. Practise it on Saturday evening.

This is one shape, not the shape. Control 7 has to work at every enforcement point at once; Control 9 runs on a schedule against a frozen baseline. Same record, same binding, different execution mode, which is the whole point of slide 14.

WORKED EXAMPLES

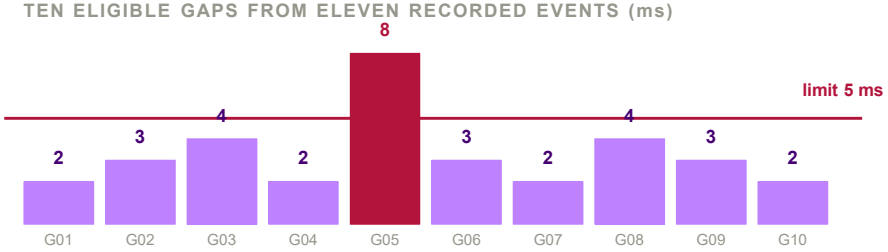
Control 7 and Control 9, with the numbers in

Illustrative values, not measured lab results. What matters is the shape: a rule declared first, a measurement, a count of exceptions, a verdict.

7 Record events, and test the gaps

THE RULE: 100% of events recorded · gap < 5 ms · violating gaps ≤ 10%

TEN ELIGIBLE GAPS FROM ELEVEN RECORDED EVENTS (ms)



SATISFIED

11 / 11 events recorded · 1 / 10 gaps violate = 10% · within the 10% tolerance

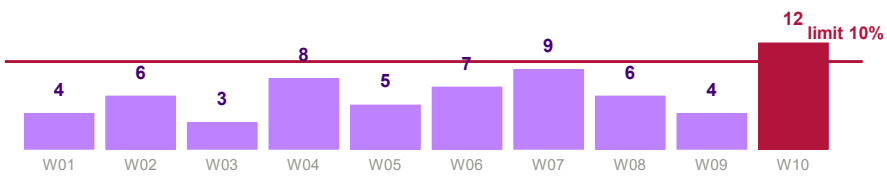
The 8 ms exception stays visible in the record. A second violating gap would give 2 / 10 = 20% and fail.

A gap test checks timing, not completeness. Define the event stream and the expected event list separately.

9 Measure drift against a frozen baseline

THE RULE: relative drop ≤ 10% · violating windows 0% · coverage 100%

RELATIVE SCORE DROP ACROSS TEN COMPARABLE WINDOWS (%)



NOT SATISFIED

W10: (0.900 − 0.792) / 0.900 × 100 = 12% · 1 / 10 windows violate, 0 allowed

The monitor may have raised the alert correctly. Detection working does not erase the breach.

A missing measurement is not zero drift. Track expected windows and data availability separately.

Each rule carries two different numbers: the limit on a single observation, and the share of observations allowed to break it. Declare both.

YOUR TASK

What you need to do

Six required steps, four ways to earn bonus points, one thing you may do if you want to, and two we are not asking for.

THE SIX REQUIRED STEPS, IN ORDER

1. **Choose your control, or controls**
   Three to choose from: 7 event recording, 9 drift, 16 spend cap. One completed control is the minimum; a second and a third are bonus.
2. **Declare your threshold ranges**
   Metric, observation limit, exception tolerance, measurement basis and a named owner. Versioned and dated before any assessed run — we check the date.
3. **Decide where in the timeline you check it**
   Post-run forensics is the floor: a tool that inspects the evidence after the fact. Doing it continuously, inline as it runs, is bonus.
4. **Build the control**
   At an enforcement point of your choosing, emitting records bound to the control, the threshold version and the run, not just a timestamp.
5. **Build the control test and / or evaluators**
   The thing that returns a verdict: recompute the measure, reconcile expected against observed, count exceptions, compare with your rule.
6. **Run it both ways, and record the gaps**
   One passing case, one breaching case. Do not move the threshold to make it pass. Write down what you could not close.

BONUS Extra points, in rough order of value

- + **A second control, then a third**
  each one completed end to end
- + **Continuous compliance**
  form the verdict inline, in the path, as it runs
- + **Adversarial testing**
  try to sabotage your own architecture, and show us what broke
- + **A second or more enforcement points**
  the same design, moved, with nothing rewritten

ALLOWED

Building more or changing the agents. You can use NVIDIA Nemotron model for this, and also to perform adversarial testing of your solution

OUT OF SCOPE Not asked for, and not scored

- Signing, hashing and Merkle trees over the ledger
- A human actually answering an alert. The response is not required; the alert, its named recipient and the silence must still be recorded

YOU ARE DONE WHEN You are done when somebody who is not on your team can ask “was control 9 satisfied for run FM-B?” and get an answer in two minutes, with the records behind it.



TRUSTWORTHY AI AND DATA GOVERNANCE HACKATHON

D. What does it take to win

Published on day one, so you can score yourself from the beginning rather than just at the end.

The team that evidences the most of the claim, with records a stranger can check and an architecture that would survive being moved to different enforcement points.

accenture

HOW YOU WILL BE JUDGED

Three axes: how many, how well, and what extra

Axis 1 orders the field, axis 2 decides the winner, axis 3 separates teams that are level. All of it is published today.

AXIS 1 · HOW MANY OF THE THREE CONTROLS DID YOU COMPLETE, END TO END?

None complete
Nothing to score

1 of 3 complete
The minimum. You are in the running

2 of 3 complete
Ahead of a team with one

3 of 3 complete
The full set

AXIS 2 · QUALITY, WHICH IS WHERE THE PRIZE IS ACTUALLY DECIDED

The live test

We name one control and one window and watch you produce the answer. Two minutes. If we have to trawl your logs, you lose the mark.

The architecture

Would it survive a second enforcement point? Is the register out of the live path? Is the evaluator stateless? Does the control hold its own threshold?

The evidence

Bound to a control, a threshold version and a run. Verifiable by somebody who is not you. Gaps recorded rather than left blank.

The honesty

“Built properly: yes. Actually worked: no” is the most useful answer a team can bring. One honest failure scores better than everything green.

AXIS 3 · BONUS POINTS, AND WHAT EARNS EACH ONE

+ Extra controls
a second or third completed to the same standard

+ Continuous compliance
a verdict formed inline, in the path, while it runs

+ Adversarial testing
you attacked your own design and showed us what broke

+ Second enforcement point
the same architecture, moved, with nothing rewritten

WHAT LOSES MARKS A threshold with no date, or dated after the test · records covering only some of the governed actions · evidence only your own team can verify · a finding closed without a passing re-test · a control that stores its own threshold · a design that only works in the request path

NEW SLIDE

JUDGEMENT DAY

What to bring, and what we will do with it

If an artefact is missing we cannot score it, however good the work behind it was. Assemble these on Saturday evening, not Sunday morning.

BRING THESE, WHATEVER YOU BUILT

1. **Your evidence file** — MUST
   In whatever form you chose. Exported, openable by us, and covering the runs you want scored.
2. **Your threshold document** — MUST
   One entry per control you took on: metric, limit, exception tolerance, measurement basis, owner, version and date.
3. **Your control test tool** — MUST
   The thing we will drive. A command line, a script, an API, a small app. We type the question; you do not narrate the answer.
4. **Two runs, named** — MUST
   One that passes and one that breaches, with their run identifiers written down before we arrive.
5. **A one-page architecture sketch** — MUST
   Components, what sits in the path of execution, and how an enforcement point invokes you. A whiteboard photograph is fine.
6. **Your gap list** — MUST
   What you could not close, and why. This is scored, and it scores well.

Bring one honest failure. A team that says "built properly, did not work, here is the evidence and here is why" scores above a team with nothing but green ticks.

BONUS Only if you went after the extra points

- + **More than one control implemented**
  More than one control implemented end to end in your architecture
- + **An inline decision, live**
  show a verdict being formed in the path, not replayed afterwards
- + **Your adversarial test log**
  what you attacked, what broke, and what you changed
- + **The second or more enforcement points**
  running, with a note of what you had to change

OUT OF SCOPE Do not spend time on these

- Signatures, hash chains or Merkle proofs over the ledger
- A slide deck. A whiteboard and a working tool beat both

THE TWO MINUTES

We name one control and one run. You drive your own tool. Rehearse it twice before you present.

TRUSTWORTHY AI AND DATA GOVERNANCE HACKATHON

Thank you!

accenture